

Lecture 10: Word embeddings

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will

1. Understand Word2Vec architectures (CBOW and Skip-gram)
2. Explain how negative sampling makes training efficient
3. Compare Word2Vec, GloVe, and FastText
4. Apply word embeddings for analogies and similarity
5. Recognize biases in word embeddings

The 2013 revolution

Word2Vec showed that neural networks can learn powerful semantic representations from raw text alone.

From count-based to prediction-based

Count-Based (LSA, LDA):

- Build co-occurrence matrix
- Apply matrix factorization
- Global statistics
- Linear relationships

Prediction-Based (Word2Vec):

- Predict context from word
- Train neural network
- Local context windows
- Non-linear relationships

Key difference

Word2Vec learns by predicting — the embeddings are a byproduct!

Word2Vec: Core intuition

Words that appear in similar contexts should have similar representations

Context window

"The quick brown **[TARGET]** jumped over the lazy dog"

Context: [The, quick, brown, jumped, over, the, lazy, dog]

Target: fox

Key Idea:

- Train a model to predict target from context (or vice versa)
- The learned **weights** become our word vectors!
- We don't care about the prediction task — we want the embeddings!

CBOW vs. Skip-gram

CBOW (Continuous Bag of Words):

- Given context → predict center word
- "the, cat, on, the" → "sat"
- Faster to train
- Better for frequent words

Skip-gram:

- Given center word → predict context
- "sat" → "the, cat, on, the"
- Slower but better quality
- Better for rare words
- **Most commonly used**

Context windows: worked example

Sentence: "The cat sat on the mat" | **Window size = 2**

1	Position 0: "The" → Context: [cat, sat]
2	Position 1: "cat" → Context: [The, sat, on]
3	Position 2: "sat" → Context: [The, cat, on, the]
4	Position 3: "on" → Context: [cat, sat, the, mat]
5	Position 4: "the" → Context: [sat, on, mat]
6	Position 5: "mat" → Context: [on, the]

Skip-gram training pairs (target → context):

1	(The, cat), (The, sat), (cat, The), (cat, sat), (cat, on), ...
---	--

The efficiency problem

Challenge: Softmax over entire vocabulary is expensive!

$$P(w_{context} | w_{center}) = \frac{\exp(v_{context}^T v_{center})}{\sum_{w=1}^V \exp(v_w^T v_{center})}$$

For 100k vocabulary: 100k exponentials per training example!

Solution: Negative Sampling

Instead of predicting across all words, create binary classification:

- **Positive:** Actual context word (label = 1)
- **Negative:** K random words (label = 0)

Training becomes $O(k)$ instead of $O(V)$!

Negative sampling example

Training pair: ("cat", "sat") — cat is center, sat is context

```
1 # Positive sample: Does "sat" appear near "cat"? YES (label=1)
2 positive_pair = ("cat", "sat", label=1)
3
4 # Negative samples: Random words that DON'T appear near "cat"
5 negative_pairs = [
6     ("cat", "algorithm", label=0),
7     ("cat", "president", label=0),
8     ("cat", "quantum", label=0),
9 ]
10
11 # Train binary classifier: Is this a real context pair?
12 # Instead of 100k-way softmax → just 4 binary predictions!
```

Typical k: 5-20 negative samples per positive

The famous result: vector arithmetic

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

Other analogies

- $\vec{\text{Paris}} - \vec{\text{France}} + \vec{\text{Italy}} \approx \vec{\text{Rome}}$
- $\vec{\text{walking}} - \vec{\text{walk}} + \vec{\text{swim}} \approx \vec{\text{swimming}}$
- $\vec{\text{bigger}} - \vec{\text{big}} + \vec{\text{small}} \approx \vec{\text{smaller}}$

Why does this work?

Subtracting "man" removes the "male" component; adding "woman" adds the "female" component. Result: "female royal" = queen!

Word2Vec in Python

```
1 from gensim.models import Word2Vec
2 import gensim.downloader as api
3
4 # Load pre-trained model (3B words, 300 dimensions)
5 model = api.load('word2vec-google-news-300')
6
7 # Find similar words
8 model.most_similar('computer', topn=5)
9 # [('computers', 0.72), ('laptop', 0.69), ('PC', 0.68), ... ]
10
11 # Word analogies: king - man + woman = ?
12 model.most_similar(positive=['king', 'woman'], negative=['man'], topn=1)
13 # [('queen', 0.71)]
14
15 # Train your own
16 sentences = [['the', 'cat', 'sat'], ['the', 'dog', 'ran']]
17 custom_model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, sg=1)
```

GloVe: Global Vectors

Combining count-based and prediction-based methods

Key Insight:

Ratios of co-occurrence probabilities encode meaning better than raw probabilities!

Probe	P(w ice)	P(w steam)	Ratio
solid	high	low	$\gg 1$
gas	low	high	$\ll 1$
water	high	high	~ 1

GloVe learns:

$$\vec{w}_i^T \vec{w}_j \approx \log P(i, j)$$

Word vectors whose dot product relates to co-occurrence probability!

GloVe vs. Word2Vec

Aspect	Word2Vec	GloVe
Approach	Local context windows	Global co-occurrence
Training	Online (stochastic)	Batch (matrix factorization)
Speed	Medium	Fast for large corpora
Quality	Excellent	Excellent

In practice

Similar performance on most tasks! Choose based on:

- Corpus size (GloVe better for very large)
- Training infrastructure (GloVe needs memory for matrix)

FastText: Subword information

The Problem: Word2Vec/GloVe give ONE vector per word

Out-of-Vocabulary (OOV) Problem

- New word? Unknown!
- Misspelling? Unknown!
- Rare morphological form? Unknown!

But "running", "runner", "runnable" share morphology!

FastText Solution: Break words into character n-grams

FastText: Character n-grams

Example: "where" (with n=3)

Character trigrams: `<wh, whe, her, ere, re>`

Word vector = average of n-gram vectors:

$$\vec{w}_{where} = \frac{1}{6}(\vec{z}_{<wh} + \vec{z}_{whe} + \vec{z}_{her} + \vec{z}_{ere} + \vec{z}_{re>} + \vec{z}_{<where>})$$

Benefits

- Handles OOV words!
- Captures morphology ("unhappiness" shares n-grams with "unhappy", "happiness")
- Great for morphologically rich languages (German, Turkish, Finnish)
- Robust to typos

Method comparison

Method	Year	OOV Handling	Best For
LSA	1990	No	Topic modeling
LDA	2003	No	Interpretable topics
Word2Vec	2013	No	General NLP
GloVe	2014	No	Large corpora
FastText	2017	Yes!	Morphology-rich languages

Typical dimensions

50-300 dimensions (100-300 most common for best quality)

Bias in word embeddings

Embeddings learn biases from training data

Gender bias examples

- $\vec{man} : \text{computer programmer} :: \vec{woman} : \text{homemaker}$
- $\vec{man} : \text{doctor} :: \vec{woman} : \text{nurse}$

```
1 def gender_bias_score(word):  
2     return model.similarity(word, 'man') - model.similarity(word, 'woman')  
3  
4 # Results from Google News Word2Vec:  
5 # programmer: +0.091 (toward man)  
6 # nurse: -0.109 (toward woman)
```

Why this matters

Embeddings used in hiring tools, search, recommendations can perpetuate bias!

Practical tips

1. **Start with pre-trained models**

- Word2Vec: Google News (300d, 3B words)
- GloVe: Common Crawl (300d, 840B tokens)
- FastText: 157 languages available

2. **Use cosine similarity** (not Euclidean distance)

$$\text{sim}(u, v) = \frac{u \cdot v}{||u|| \cdot ||v||}$$

3. **Be aware of biases** — test for bias, consider debiasing techniques

4. **Remember limitations** — static (one vector per word), no polysemy

Summary

What we learned:

1. **Word2Vec (2013)**: Neural revolution — predict context from words
 - Skip-gram most common, negative sampling for efficiency
2. **GloVe (2014)**: Global co-occurrence statistics
3. **FastText (2017)**: Character n-grams handle OOV
4. **Bias**: Embeddings reflect training data biases

Key limitation

Static embeddings: one vector per word type

Next week: Contextual embeddings (ELMo, BERT) solve polysemy!

Key references

Foundational Papers:

- Mikolov et al. (2013). "Efficient Estimation of Word Representations"
- Pennington et al. (2014). "GloVe: Global Vectors for Word Representation"
- Bojanowski et al. (2017). "Enriching Word Vectors with Subword Information"

Tools:

- Gensim: `Word2Vec` , `FastText` | Pre-trained:
`gensim.downloader`

Assignment 3: Wikipedia Embeddings

Apply this week's concepts:

- Train embeddings on Wikipedia data
- Compare LSA vs. Word2Vec
- Explore semantic relationships
- Visualize with UMAP

Think about:

- How does corpus size affect quality?
- What analogies work? What fails?
- Can you detect bias?

Link

[Assignment 3: Wikipedia Embeddings](#)

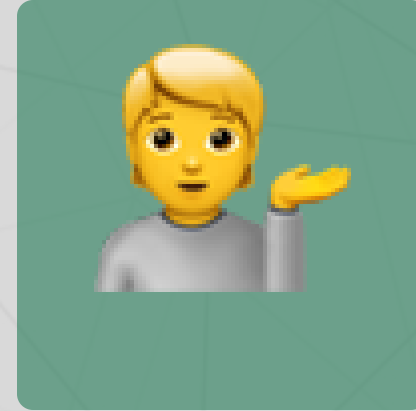
Questions?



Email me



Join our Discord



Come to office hours

Week 3 complete!

Next week: Contextual embeddings and dimensionality reduction!